

Product Review and Rating Management System

1.Review

Attribute	Key	Description
Review_ID	Primary Key (PK)	Unique review ID
Rating	-	Customer rating (1–5 stars)
Comment	-	Customer feedback about the product
Review_Date	-	Date the review was submitted
Customer_ID	Foreign Key (FK)	ID of the customer who wrote the review
Product_ID	Foreign Key (FK)	ID of the reviewed product

```
SQL> CREATE TABLE Review3 (  
2     Review_ID NUMBER PRIMARY KEY,  
3     Rating NUMBER(1) CHECK (Rating BETWEEN 1 AND 5),  
4     Review_Comment VARCHAR2(500),  
5     Review_Date DATE,  
6     Customer_ID NUMBER,  
7     Product_ID NUMBER  
8 );
```

```
SQL> INSERT INTO Review3 VALUES  
2     (1, 5, 'Excellent product', DATE '2026-10-01', 101, 1001);  
1 row created.
```

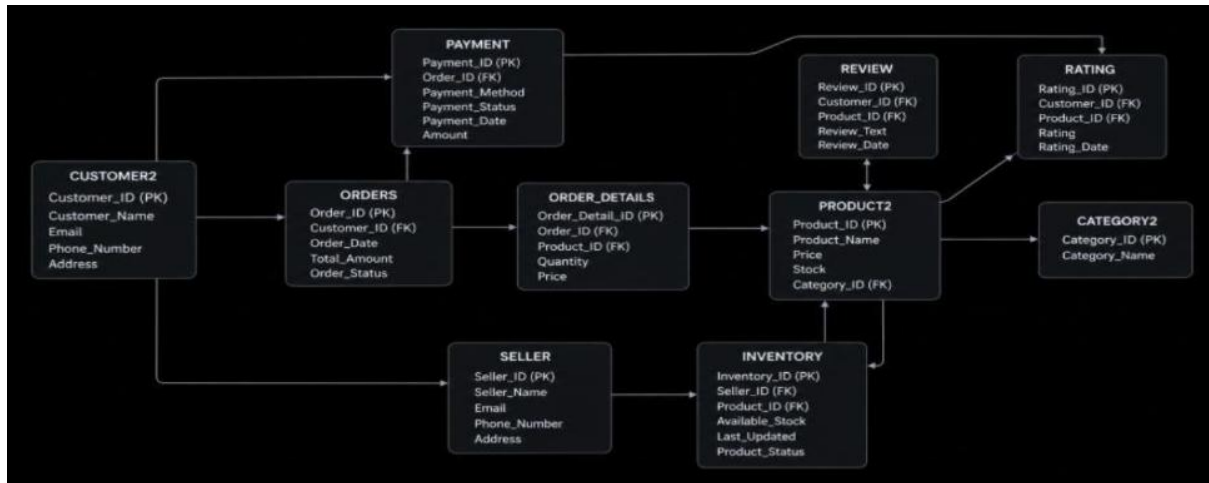
```
SQL>  
SQL> INSERT INTO Review3 VALUES  
2     (2, 4, 'Good quality', DATE '2026-10-02', 102, 1002);  
1 row created.
```

```
SQL>  
SQL> INSERT INTO Review3 VALUES  
2     (3, 3, 'Average product', DATE '2026-10-03', 103, 1003);  
1 row created.
```

```
SQL>  
SQL> INSERT INTO Review3 VALUES  
2     (4, 5, 'Very good product', DATE '2026-10-04', 104, 1004);  
1 row created.
```

```
SQL>  
SQL> INSERT INTO Review3 VALUES  
2     (5, 4, 'Worth the price', DATE '2026-10-05', 105, 1005);  
1 row created.
```

2. Table Relationship



3. Store customer feedback and ratings

```

SQL> SELECT Review_ID,
2         Product_ID,
3         Customer_ID,
4         Rating,
5         Review_Comment,
6         Review_Date
7 FROM Review3;
  
```

REVIEW_ID	PRODUCT_ID	CUSTOMER_ID	RATING
REVIEW_COMMENT			
REVIEW_DA			
1	1001	101	5
Excellent product			
01-OCT-26			
2	1002	102	4
Good quality			
02-OCT-26			
REVIEW_ID	PRODUCT_ID	CUSTOMER_ID	RATING
REVIEW_COMMENT			
REVIEW_DA			
3	1003	103	3
Average product			
03-OCT-26			
4	1004	104	5
Very good product			
REVIEW_ID	PRODUCT_ID	CUSTOMER_ID	RATING
REVIEW_COMMENT			
REVIEW_DA			
04-OCT-26			
5	1005	105	4
Worth the price			
05-OCT-26			

4. Rating Table

Attribute	Key	Description
Rating_ID	Primary Key (PK)	Unique rating ID
Customer_ID	Foreign Key (FK)	ID of the customer who gave the rating
Product_ID	Foreign Key (FK)	ID of the product being rated
Rating	-	Customer rating from 1 to 5 stars
Rating_Date	-	Date on which the rating was submitted

```
SQL> CREATE TABLE Rating3 (  
2     Rating_ID NUMBER PRIMARY KEY,  
3     Review_ID NUMBER,  
4     Rating NUMBER(1) CHECK (Rating BETWEEN 1 AND 5),  
5     FOREIGN KEY (Review_ID) REFERENCES Review3(Review_ID)  
6 );
```

Table created.

```
SQL> INSERT INTO Rating3 VALUES (1, 1, 5);  
1 row created.  
SQL> INSERT INTO Rating3 VALUES (2, 2, 4);  
1 row created.  
SQL> INSERT INTO Rating3 VALUES (3, 3, 3);  
1 row created.  
SQL> INSERT INTO Rating3 VALUES (4, 4, 5);  
1 row created.  
SQL> INSERT INTO Rating3 VALUES (5, 5, 4);  
1 row created.
```

5.Retrieve Product Review Details

```
SQL> SELECT * FROM Review3;
```

REVIEW_ID	RATING	REVIEW_COMMENT
1	5	Excellent product
01-OCT-26	101	1001

2	4	Good quality
02-OCT-26	102	1002

REVIEW_ID	RATING	REVIEW_COMMENT
3	3	Average product
03-OCT-26	103	1003

4	5	Very good product
---	---	-------------------

REVIEW_ID	RATING	REVIEW_COMMENT
04-OCT-26	104	1004
5	4	Worth the price
05-OCT-26	105	1005

7. Calculate Average Product Rating

```
SQL> SELECT Product_ID,  
2         AVG(Rating) AS Average_Rating  
3 FROM Review3  
4 GROUP BY Product_ID;
```

PRODUCT_ID	AVERAGE_RATING
1001	5
1002	4
1003	3
1004	5
1005	4

8. Identify Highly Rated Products

```
SQL> SELECT Product_ID,  
2         AVG(Rating) AS Average_Rating  
3 FROM Review3  
4 GROUP BY Product_ID  
5 HAVING AVG(Rating) >= 4;
```

PRODUCT_ID	AVERAGE_RATING
1001	5
1002	4
1004	5
1005	4